



UNIVERSIDAD DE LAS FUERZAS ARMADAS

ESPE

Departamento de Ciencias de la Computación.

INTEGRANTES: Cristian Suquillo
Joshep Chisaguano.
Melanie Suñiga.
Paul Villacis.

FECHA: 21 de mayo del 2026.

GRUPO: Grupo 5

ASIGNATURA: Metodología de desarrollo de Software.

DOCENTE: Mgtr. Jenny Ruiz.

NRC: 30693

Índice

TALLER DE MODELAMIENTO DE CRUD PARA ESTUDIANTES	3
1. CASOS DE USO.....	3
1.1 Creación de Registros: Agregar Estudiante	3
1.2 Modificación de Registros: Actualizar Estudiante	3
1.3 Depuración de Datos: Eliminar Estudiante	4
1.4 Consulta General: Mostrar Todo	4
2. REQUISITOS FUNCIONALES.....	6
2.1 Documento de Ingeniería de Requerimientos y Trazabilidad UML.....	6
2.2 RF-01: Agregar estudiante (Trazabilidad: UC1)	6
2.3 RF-02: Actualizar estudiante (Trazabilidad: UC2).....	6
2.4 RF-03: Eliminar estudiante (Trazabilidad: UC3).....	6
2.5 RF-04: Mostrar todo (Trazabilidad: UC4)	7
2.6 RF-05: Validar datos del estudiante (Trazabilidad: UC5).....	7
2.7 Análisis de la Matriz de Desarrollo.....	7
3. DIAGRAMA DE CASOS DE USO	9
4. DIAGRAMA DE CLASES	11
5. CONCLUSIÓN TÉCNICA.	17
5.1 Reflexión breve sobre trazabilidad.....	17
5.2 Consistencia	17
5.3 Utilidad del modelo	17
6. REFERENCIAS.	18
7. RUBRICA.....	18

Taller de Modelamiento de CRUD para Estudiantes

1. Casos de Uso.

Este módulo contiene las cuatro operaciones fundamentales para la administración del ciclo de vida de los datos de los estudiantes. Todas las acciones descritas a continuación son ejecutadas de manera exclusiva por el actor Administrador Académico.

1.1 Creación de Registros: Agregar Estudiante

Esta función permite dar de alta a un nuevo alumno en la base de datos del sistema.

Condición Previa (Precondición): Para evitar la duplicidad de información, el sistema debe verificar que el ID ingresado no corresponda a ningún estudiante registrado previamente.

Flujo del Proceso: El administrador interactúa con la interfaz ingresando tres datos obligatorios: el ID (identificador único), el Nombre y la Edad. Posteriormente, el sistema ejecuta de forma automática una rutina de validación de datos (verificando formatos y campos vacíos). Si la validación es exitosa, se procede a guardar el registro.

Resultado Final (Postcondición): El sistema confirma visualmente que el estudiante ha sido agregado correctamente y sus datos quedan disponibles para futuras consultas.

1.2 Modificación de Registros: Actualizar Estudiante

Permite corregir o actualizar la información de un alumno que ya forma parte de la institución.

Condición Previa (Precondición): A diferencia del proceso de alta, aquí es un requisito indispensable que el estudiante ya exista en la base de datos con el ID proporcionado.

Flujo del Proceso: El administrador ingresa o selecciona el ID del estudiante que desea modificar. La interfaz habilitará los campos para editar el ID, Nombre y/o la Edad. Una vez realizados los cambios, el sistema vuelve a validar los nuevos datos antes de guardar los cambios de manera definitiva.

Resultado Final (Postcondición): El sistema sobrescribe la información antigua y confirma que los datos del estudiante han sido actualizados con éxito.

1.3 Depuración de Datos: Eliminar Estudiante

Esta operación se encarga de dar de baja o remover el registro de un alumno del sistema.

Condición Previa (Precondición): El estudiante debe estar agregado. Además, el sistema debe verificar que se cumpla cualquier regla de negocio específica para la eliminación (por ejemplo, que el alumno no tenga materias activas o deudas pendientes).

Flujo del Proceso: El administrador busca al estudiante mediante la selección o ingreso de su ID. Para prevenir pérdidas accidentales de información, el sistema solicita una confirmación de eliminación. Una vez aprobada, se ejecuta la acción de eliminar el registro.

Resultado Final (Postcondición): Dependiendo de las políticas de TI, el estudiante es eliminado (borrado lógico) para conservar el historial, quedando inactivo para el resto de los módulos.

1.4 Consulta General: Mostrar Todo

Función diseñada para la auditoría, revisión y visualización masiva de la información almacenada.

Condición Previa (Precondición): El sistema requiere que se hayan agregado estudiantes para mostrar información o, en su defecto, que la estructura de la tabla esté disponible y lista para ser renderizada, aunque se encuentre vacía.

Flujo del Proceso: El administrador simplemente presiona o selecciona la opción "Mostrar". El sistema realiza una consulta interna para recuperar los registros y procede a presentar la tabla en la pantalla del usuario.

Resultado Final (Postcondición): Se despliega un listado completamente visible y estructurado en columnas que expone el ID, Nombre y Edad de todos los estudiantes del sistema.

Caso de uso	Actor	Precondición	Flujo principal resumido	Resultado esperado
Agregar estudiante	Administrador académico	El estudiante no debe existir previamente con el mismo ID.	Ingresar ID, Nombre y Edad -> validar datos -> guardar registro.	Estudiante agregado correctamente.
Actualizar estudiante	Administrador académico	El estudiante debe existir.	Ingresar o seleccionar ID -> modificar Nombre y/o Edad -> validar datos -> guardar cambios.	Datos del estudiante actualizados.
Eliminar estudiante	Administrador académico	El estudiante debe existir y cumplir regla para eliminación.	Seleccionar o ingresar ID -> confirmar eliminación -> eliminar registro.	Estudiante eliminado o marcado como eliminado.
Mostrar todo	Administrador académico	Debe existir al menos un registro o una tabla vacía disponible.	Seleccionar Mostrar Todo -> mostrar registros -> presentar tabla.	Listado visible con ID, Nombre y Edad.

2. Requisitos Funcionales.

Este apartado funciona como una Matriz de Trazabilidad de Requerimientos. A diferencia del enfoque de procesos anterior, este documento conecta formalmente las necesidades del negocio (Requerimientos Funcionales) con las reglas técnicas que el equipo de desarrollo debe cumplir (Criterios de Aceptación) y su representación en el diseño visual (Casos de Uso de PlantUML).

2.1 Documento de Ingeniería de Requerimientos y Trazabilidad UML

El propósito de este documento es definir el alcance técnico, la criticidad y las reglas de validación para el desarrollo del módulo CRUD de Estudiantes. Cada requerimiento está directamente vinculado a un identificador de modelado para garantizar que el código final coincida exactamente con el diseño arquitectónico.

2.2 RF-01: Agregar estudiante (Trazabilidad: UC1)

Actor Originador: Administrador académico.

Prioridad del Negocio: Alta. Es un bloque central e indispensable para iniciar la operación del módulo.

Criterio de Aceptación (Garantía de calidad): Para que esta función se considere terminada y correcta, el sistema debe asegurar de forma estricta que no se dupliquen identificadores (el ID debe ser único), que el campo "Nombre" no se envíe en blanco y que la "Edad" cumpla con los rangos numéricos permitidos por la institución.

2.3 RF-02: Actualizar estudiante (Trazabilidad: UC2)

Actor Originador: Administrador académico.

Prioridad del Negocio: Alta. Modificar la información existente es crítico para mantener la fidelidad de los datos en el tiempo.

Criterio de Aceptación (Garantía de calidad): El sistema permitirá la edición de los campos de Nombre y/o Edad únicamente si la petición pasa primero por el filtro de validación de campos obligatorios. No se guardarán cambios parciales o corruptos.

2.4 RF-03: Eliminar estudiante (Trazabilidad: UC3)

Actor Originador: Administrador académico.

Prioridad del Negocio: Media. Aunque es una función CRUD básica, su implementación puede postergarse brevemente en favor del alta y la edición, o manejarse mediante restricciones controladas.

Criterio de Aceptación (Garantía de calidad): Al ejecutar la orden, el software debe proceder a la eliminación física del registro o, en su defecto, aplicar un "borrado lógico" (marcarlo como inactivo) para preservar la integridad referencial del estudiante seleccionado.

2.5 RF-04: *Mostrar todo (Trazabilidad: UC4)*

Actor Originador: Administrador académico.

Prioridad del Negocio: Alta. La visualización de los datos es vital para la toma de decisiones y la auditoría interna.

Criterio de Aceptación (Garantía de calidad): La respuesta del sistema ante esta petición debe ser una interfaz tabular estructurada, la cual expondrá de manera obligatoria y ordenada tres columnas específicas: ID, Nombre y Edad.

2.6 RF-05: *Validar datos del estudiante (Trazabilidad: UC5)*

Actor Originador: Sistema (Proceso automatizado de backend).

Prioridad del Negocio: Alta. Funciona como la barrera de seguridad y calidad de todo el módulo.

Criterio de Aceptación (Garantía de calidad): Actuando como un escudo interceptor, el sistema debe bloquear activamente cualquier intento de inserción o actualización si detecta un ID vacío, un nombre sin caracteres o una edad que no corresponda a un valor válido. Emitirá un error en lugar de procesar la solicitud.

2.7 *Análisis de la Matriz de Desarrollo*

Enfoque de Prioridades: El desarrollo debe enfocarse en una primera fase "Fast-Track" para resolver los requerimientos con prioridad **Alta** (RF-01, RF-02, RF-04, RF-05), dejando el RF-03 (Eliminar) para el cierre del sprint debido a su prioridad **Media**.

Evidencia de Modelado: Los códigos de evidencia (UC1 a UC5) sirven como el puente directo hacia el diagrama de casos de uso de PlantUML. Cualquier cambio en los criterios de aceptación de esta tabla obligará a una actualización en la lógica del diagrama correspondiente.

RF	Caso de uso	Actor	Prioridad	Criterio de aceptación	Evidencia PlantUML
RF-01	Agregar estudiante	Administrador académico	Alta	El sistema guarda un estudiante con ID único, Nombre obligatorio y Edad válida.	UC1
RF-02	Actualizar estudiante	Administrador académico	Alta	El sistema modifica Nombre y/o Edad luego de validar campos obligatorios.	UC2
RF-03	Eliminar estudiante	Administrador académico	Media	El sistema elimina o marca como eliminado al estudiante seleccionado.	UC3
RF-04	Mostrar todo	Administrador académico	Alta	El sistema muestra una tabla con las columnas ID, Nombre y Edad.	UC4
RF-05	Validar datos del estudiante	Sistema	Alta	El sistema impide guardar o actualizar registros con ID vacío, nombre vacío o edad inválida.	UC5

3. Diagrama de Casos de Uso

@startuml

left to right direction

skinparam packageStyle rectangle

skinparam actorStyle awesome

actor "Administrador Académónico" as Admin

rectangle "Sistema de gestión de estudiantes" {

 usecase "Agregar estudiante" as UC1

 usecase "Actualizar estudiante" as UC2

 usecase "Eliminar estudiante" as UC3

 usecase "Mostrar todos" as UC4

 usecase "Validar datos" as UC5

}

' Conexiones del administrador

Admin --> UC1 : ingresa datos

Admin --> UC2 : busca por ID

Admin --> UC3 : busca por ID

Admin --> UC4

' Relaciones de inclusión para validación

UC1 ..> UC5 : <<include>>

UC2 ..> UC5 : <<include>>

' Notas explicativas

note right of UC1

****Campos obligatorios:****

--

*** **ID**** (Identificador único)

*** **Nombre**** (Texto completo)

*** **Edad**** (Numérico entero)

end note

note right of UC2

Solicita ID, muestra datos

actuales y permite editar

nombre y edad

end note

note right of UC3

Solicita ID y pide

confirmación antes

de eliminar

end note

note right of UC5

****Validaciones:****

--

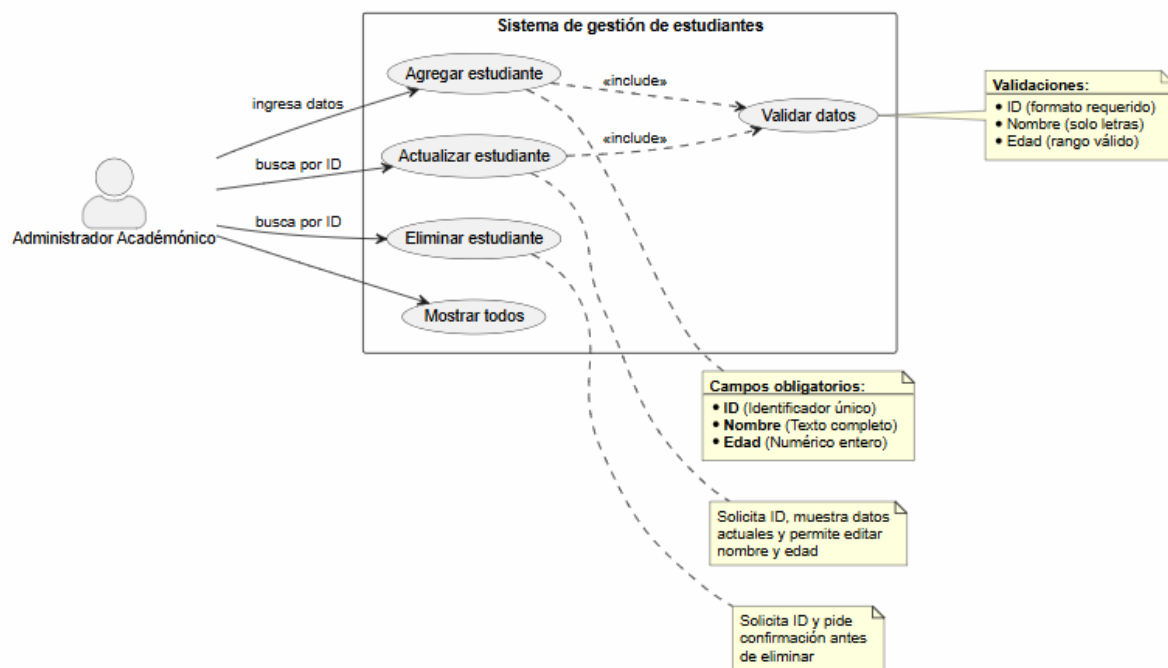
* ID (formato requerido)

* Nombre (solo letras)

* Edad (rango válido)

end note

@enduml



4. Diagrama de Clases

@startuml

skinparam classAttributeIconSize 0

skinparam shadowing false

skinparam packageStyle rectangle

skinparam linetype ortho

skinparam nodesep 80

skinparam ranksep 80

hide empty members

top to bottom direction

rectangle "SISTEMA MVC - GESTIÓN DE ESTUDIANTES" {

package "MODELO" #LightBlue {

class Estudiante {

- nombre : String

- apellido : String

- id : String

- edad : int

--

+ Estudiante()

+ getNombre() : String

+ setNombre() : void

+ getApellido() : String

+ setApellido() : void

+ getId() : String

+ setId() : void

+ getEdad() : int

+ setEdad() : void

}

class Datos {

- lista_estudiantes : ArrayList

--

+ agregarEstudiante() : void

```

    + getLista_estudiantes() : ArrayList
}

class logica_repositorio {

    - datos : Datos

    --

    + existeId() : boolean

    + buscarPorId() : Estudiante

    + validarNombre() : boolean

    + validarApellido() : boolean

    + validarEdad() : boolean

    + validarId() : boolean

}

}

package "VISTA" #LightGreen {

interface interfaz_estudiante {

    + getBtnAgregar()

    + getBtnActualizar()

    + getBtnEliminar()

    + getBtnMostrar()

    + limpiarCampos()

    + getId()

    + getNombre()

    + getApellido()

```

```
+ getEdad()

}

class Formulario_estudiante {

    - datos : Datos

    --

    + Formulario_estudiante()

    + limpiarCampos() : void

    + getTablaEstudiantes()

}

}

package "CONTROLADOR" #LightYellow {

class controlador_estudiante {

    - ver : interfaz_estudiante

    - datos : Datos

    - logica : logica_repositorio

    --

    + agregar() : void

    + actualizar() : void

    + eliminar() : void

    + mostrarTabla() : void

}

}
```

```
}
```

```
'=====
```

```
' RELACIONES
```

```
'=====
```

```
Datos *-- Estudiante : contiene
```

```
logica_repositorio --> Datos : usa
```

```
logica_repositorio --> Estudiante : valida
```

```
Formulario_estudiante ..|> interfaz_estudiante
```

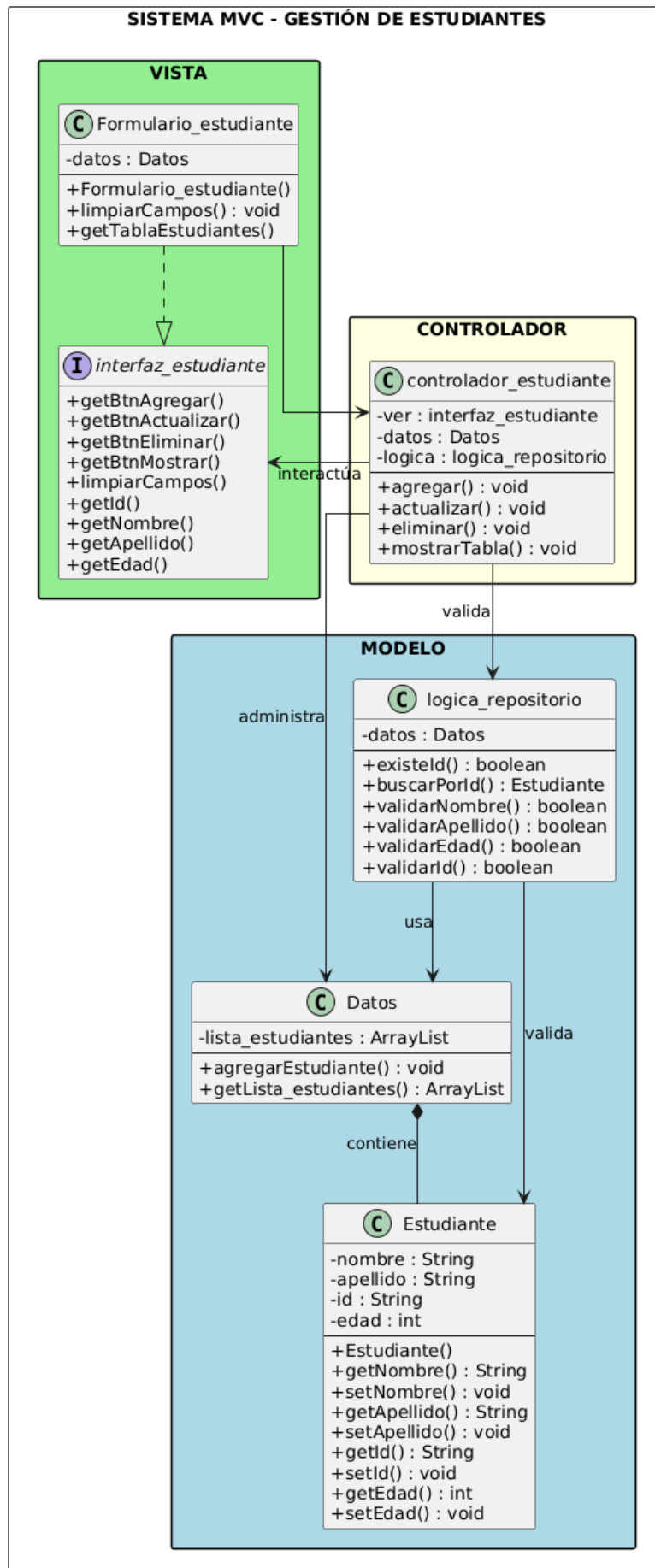
```
controlador_estudiante --> interfaz_estudiante : interactúa
```

```
controlador_estudiante --> Datos : administra
```

```
controlador_estudiante --> logica_repositorio : valida
```

```
Formulario_estudiante --> controlador_estudiante
```

```
@enduml
```



5. Conclusión Técnica.

5.1 Reflexión breve sobre trazabilidad

La trazabilidad no es un simple ejercicio burocrático de marcar casillas; es el hilo conductor que transforma una lista abstracta de deseos del cliente en software funcional y seguro. Al introducir múltiples actores con diferentes niveles de acceso —como el Administrador Académico y el Estudiante— la trazabilidad deja de ser lineal y se convierte en una matriz multidimensional. Su verdadera magia radica en que actúa como un sistema de control de identidad prematuro: nos obliga a responder el "*¿quién?*" antes de empezar a programar el "*¿cómo?*", evitando que construyamos código huérfano o, peor aún, brechas de seguridad donde un actor termine ejecutando acciones que no le corresponden.

5.2 Consistencia

Un sistema consistente es aquel que no se contradice a sí mismo. La integración del modelo de requerimientos con el diagrama de casos de uso asegura que exista una correlación exacta de 1 a 1 entre lo que el negocio pide y lo que la arquitectura diseña. Si el requerimiento funcional dice que la validación es obligatoria para el alta y la edición, el diagrama de PlantUML lo refleja con precisión mediante sus relaciones de inclusión (`<<include>>`), y la base de datos lo replica con sus restricciones. Esta alineación total elimina los "teléfonos rotos" en el equipo de desarrollo, garantizando que el backend, la interfaz de usuario y las reglas de negocio hablen exactamente el mismo idioma.

5.3 Utilidad del modelo

En última instancia, el valor real de este modelo radica en su capacidad para servir como un plano arquitectónico interactivo. Para los desarrolladores, funciona como una guía de desarrollo que dicta los límites de los roles y los flujos lógicos de las operaciones CRUD. Para los líderes de proyecto, es una herramienta de estimación y control de cambios: si mañana se decide cambiar las reglas de visualización de los estudiantes, el modelo permite predecir instantáneamente qué requerimientos, qué pantallas y qué APIs se verán afectadas. En lugar de adivinar el impacto en miles de líneas de código, el modelo nos lo muestra a simple vista en un solo golpe de pantalla.

6. Referencias.

Ruiz Robalino, J. A., y Lucio Moreno, X. I. (2024). *Fundamentos de la ingeniería de requisitos. En Fundamentos de Ingeniería de Software un enfoque práctico* (Cap. 4, pp. 39-48). Universidad de las Fuerzas Armadas ESPE.

7. Rubrica

Criterio	Descripción	Puntaje	Evidencia
Trazabilidad RF-CU	Relación clara entre requisitos funcionales, actor, caso de uso y criterio de aceptación.	5 pts	Matriz RF-CU completa.
Modelado de casos de uso	Representación correcta de actor, límite del sistema, casos de uso y relaciones.	5 pts	Diagrama PlantUML generado.
Claridad del análisis	Selección pertinente de funcionalidades de nivel 0 y nombres orientados a objetivos del usuario.	4 pts	Casos de uso redactados claramente.
Uso correcto de PlantUML	Código funcional, ordenado y comprensible.	3 pts	Código sin errores de sintaxis.
Presentación técnica	Documento ordenado, con evidencias, explicación breve y coherencia visual.	3 pts	Entrega final documentada.